

Exercise 02b: Sets and Duplicates

In this exercise, we will be implementing the *Set* data structure (see `StringSet` interface) and then using it to find words in texts that occur at least twice.

Preparation

From now on we are going to use following git process for the exercises:

- Fork the exercise project under your own namespace (your username)
- Clone the forked repository, by using IntelliJ to import the project from Git, or by doing so "manually" in the console:

```
git clone LINK TO YOUR FORKED REPOSITORY
```
- After finishing the exercise `add`, `commit` and `push` your solution.

Task 1: StringSet

Complete the `StringSetImpl` class by implementing the `add`, `contains`, `remove` and `size` methods.

- Implement the *Set* as a tree structure, as discussed in the lecture.
- Implement the required helper class as an *inner class*.
- The `String` class provides the `equals` and `compareTo` methods for comparing strings, see <https://docs.oracle.com/javase/9/docs/api/java/lang/String.html>
- Verify that the `testStringSet` test runs without errors.

Task 2: Duplicates

Implement the static method `Duplicates.findDuplicates()` in which you now use the *Set* to find the words in a string that occur at least twice.

- The `String.split` method splits a single string into an array of multiple strings.
- The `String.replaceAll` method can be used to remove certain characters from a string; You may use `replaceAll("[^\\w\\s]", "")` to remove all non-word/non-whitespace characters. And `replaceAll("\\s+", " ")` to reduce sequences of multiple whitespaces to a single whitespace.
Existing strings are unchangeable (*immutable*) in Java - see the documentation <https://docs.oracle.com/javase/9/docs/api/java/lang/String.html#replaceAll-java.lang.String-java.lang.String->
- Verify that the `testFindDuplicates` test runs without errors.